Classe StringBuilder

A classe `StringBuilder` faz parte do pacote `java.lang`. Essa classe permite criar e manipular dados de Strings dinamicamente, ou seja, podem criar variáveis de String modificáveis.

O significado da classe StringBuilder não tem o mesmo sentido que classe String. Existe uma diferença entre essas classes, veja algumas características.

Características StringBuilder

- Armazena caracteres especificados pela sua capacidade, caso ela seja excedida, é aumentada para acomodar os caracteres adicionais;
- Não precisa alocar novos objetos quando realiza uma concatenação;
- Não são sincronizadas;
- Não são seguras para threads;

Uma vantagem sobre a classe String é a concatenação de strings. Pois quando concatena strings com StringBuilder é invocado o método append. Esse método aloca novas strings concatenadas para o mesmo objeto, ou seja, cada vez que concatena strings não são criadas cópias dos objetos como é realizado pelo método `concat` da classe String, contribuindo para um melhor desempenho do sistema.

Listagem 1: Concatenação de Strings com método `append`.

```
public class TestaStringBuilder {
         public static void main(String[] args) {

  StringBuilder nomes = new StringBuilder();
    nomes.append("Carlos>").append("Maria>").append("José>").append("Renata");

  System.out.println(nomes.toString());

        }
}
```

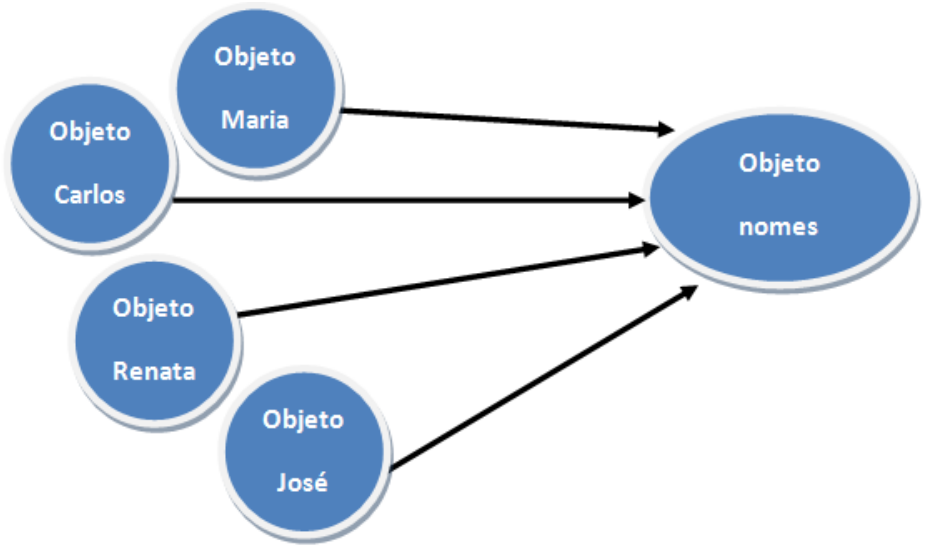


Figura 1: Ilustração gráfica do código da Listagem 1.

Métodos StringBuilder

Abaixo são apresentados os métodos principais e mais utilizados.

- length - Retorna o número de caracteres atualmente em um StringBuilder;
- capacity – Retorna o número de caracteres que pode ser armazenado em um StringBuilder sem alocar mais memória;
- ensureCapacity – Garante que um StringBuilder tenha pelo menos a capacidade especificada;
- setLength – Aumenta ou diminui o comprimento de uma StringBuilder;
- charAt – Aceita um argumento inteiro que representa o índice e retorna o caractere nessa posição no StringBuilder;
- setCharAt – Copia caracteres de um StringBuilder no array de caracteres passado como um argumento, tendo aceitação de até 4 argumentos;
- o índice inicial a partir do qual os caractere(s) devem ser copiados do StringBuilder;
- o índice um a mais do último caractere que será copiado a partir do StringBuilder;
- o array de caracteres para onde os caracteres serão copiados;
- localização inicial no array de caracteres em que o primeiro caractere deve ser colocado;

getChars – Retorna o caractere especificado;

reverse – Retorna os caracteres invertidos;

Listagem 2: Métodos capacity, ensureCapacity, setLength e length.

```
public class Metodos_StringBuilder {

  public static void main(String[] args) {

      StringBuilder buffer = new StringBuilder("João Silveira Batista");
      System.out.printf(" buffer = %s\n tamanho = %d\n capacidade (%d
```

```
(tamanho) + " +
        "16(valor inicial padrão)) = %d\n\n",
        buffer.toString(),buffer.length(),buffer.length(),buffer.capacity());

    //EXPANDE A CAPACIDADE P/ 75 CARACTERES
    buffer.ensureCapacity(75);
    System.out.printf("Nova Capacidade = %d\n\n", buffer.capacity());

    //REDEFINE O TAMANHO, NÃO APARECENDO OS CARACTERES
    // SE O ARGUMENTO FOR MENOR QUE A QUANTIDADE DE CARACTERES ORIGINAL
```

Listagem 3: Métodos charAt, setChar e reverse.

```
public class Metodos2_StringBuilder {

  public static void main(String[] args) {

    StringBuilder buffer = new StringBuilder("Usando outros métodos
StringBuilder");

    System.out.printf("buffer = %s\n", buffer.toString());
    System.out.printf("Caractere index 0: %s\nCaractere index 4: " +
        "%s\n\n",buffer.charAt(0),buffer.charAt(4));

    char[] charArray = new char[buffer.length()];

    buffer.getChars(0, buffer.length(), charArray, 0);
    System.out.print("Os caracteres são: ");

    for(char c : charArray)
```

Construtores StringBuilder

Nessa classe existem quatro tipos de construtores que são apresentados abaixo, lembrando que quando é declarado o construtor sem argumentos a capacidade inicial é identificada com 16 caracteres.

Listagem 4: Construtores.

```
public class Contrutores_StringBuilder {
  public static void main(String[] args) {
    StringBuilder buffer1 = new StringBuilder();
    StringBuilder buffer2 = new StringBuilder(5655);
    StringBuilder buffer3 = new StringBuilder("Texto Qualquer");
```

```
        System.out.printf("buffer1 = \"%s\"\n", buffer1.toString());
        System.out.printf("buffer2 = \"%s\"\n", buffer2.toString());
        System.out.printf("buffer3 = \"%s\"\n", buffer3.toString());
      }
    }
```

Método insert

Os métodos `insert` sobrecarregados inserem valores de vários tipos (primitivos, arrays de caracteres, Strings, Objects e CharSequences) em qualquer posição de um StringBuilder.

Existem alguns cuidados nesses tipos de operação, pois se inserir um número maior que um comprimento da StringBuilder ou menor que 0, irá estourar uma exceção `StringIndexOutBoundsException`.

Listagem 5: Método insert.

```
public class StringBuilder_Insert {
  public static void main(String[] args) {
    Object objetoRef = "Maria";
    String String = "Joaquina";
    char[] conjArray = {'S','i','l','v','a'};

    StringBuilder buffer = new StringBuilder();
    buffer.insert(0, objetoRef);
    buffer.insert(0, " ");
    buffer.insert(0, String);
    buffer.insert(0, " ");
    buffer.insert(0, conjArray);
    buffer.insert(0, " ");

    System.out.printf("Valores:\n%s\n\n", buffer.toString());
  }
}
```

Método delete

Os métodos `delete` excluem caracteres em qualquer posição em um StringBuilder, aceitando 2 argumentos sendo que um deles é o índice inicial e o índice a mais do fim dos caracteres a excluir. O método `deleteCharAt` aceita um argumento que é o índice do caractere a excluir.

Listagem 6: Método delete.

```
public class StringBuilder_Delete {
  public static void main(String[] args) {
    Object objetoRef = "Maria";
```

```
        String String = "Joaquina";
        char[] conjArray = {'S','i','l','v','a'};

        StringBuilder buffer = new StringBuilder();

        buffer.insert(0, objetoRef);
        buffer.insert(0, " ");
        buffer.insert(0, String);
        buffer.insert(0, " ");
        buffer.insert(0, conjArray);
        buffer.insert(0, " ");
        System.out.printf("Valores antes do delete:\n%s\n\n", buffer.toString());

        buffer.delete(0, 2)
```

A classe **StringBuilder** em Java serve para **modificar textos de forma dinâmica e eficiente**, evitando o desperdício de memória gerado pela imutabilidade da classe tradicional `String`. [1, 2]

Por que usar a classe StringBuilder?

- **Imutabilidade da String:** No Java, objetos do tipo `String` são imutáveis. Cada vez que você faz uma concatenação (ex: `texto = texto + "a"`), o Java cria um **novo objeto** na memória e descarta o antigo. [1, 2]
- **Gasto de memória (Lixo):** Em laços de repetição (`for` / `while`) ou em grandes volumes de texto, concatenar com `+` gera dezenas ou centenas de objetos temporários que sobrecarregam o *Garbage Collector*. [1, 2]
- **Mutabilidade do StringBuilder:** O **StringBuilder** é um objeto mutável. As alterações (como acréscimos e edições) ocorrem diretamente na mesma instância de memória, garantindo **alta performance**. [1, 2]
- **Thread-safety:** Diferente do `StringBuffer`, o **StringBuilder não** é seguro para múltiplas threads, mas é mais rápido quando executado em uma única linha de execução (cenário mais comum). [1, 2]

Como usar a classe StringBuilder?

1. Declaração e instanciação

Você pode criar um **StringBuilder** vazio (com capacidade inicial para 16 caracteres) ou passando um texto inicial: [1, 2]

java

```java
// Vazio (capacidade padrão de 16 caracteres)
StringBuilder sb = new StringBuilder();

// Iniciando com um texto
StringBuilder sbComTexto = new StringBuilder("Olá");
```

Use o código com cuidado.

2. Principais métodos

- **`append()`**: Adiciona (concatena) textos, números ou outros tipos ao final do **StringBuilder**.
- **`insert()`**: Insere um texto em uma posição específica (índice).
- **`delete()`**: Remove um trecho baseado em um índice inicial e final.
- **`reverse()`**: Inverte a ordem dos caracteres.
- **`toString()`**: Converte o **StringBuilder** de volta para um objeto do tipo `String` comum. [1, 2, 3, 4, 5]

3. Exemplo prático de código

java

```java
StringBuilder sb = new StringBuilder();

// Adicionando textos de forma otimizada
sb.append("Java ");
sb.append("é ");
sb.append("muito ");
sb.append("eficiente ");
sb.append("com ");
sb.append("StringBuilder!");

// Convertendo para String para exibir ou utilizar
String resultadoFinal = sb.toString();
System.out.println(resultadoFinal);
// Saída: Java é muito eficiente com StringBuilder!
```

Use o código com cuidado.

Gostaria de ver um **exemplo prático comparando o tempo de execução (benchmark)** entre concatenar com `String` e usar o **StringBuilder** em um laço `for` grande?